How State-Transition Fidelity Affects Financial Agent Evaluation

Anonymous Authors

Abstract

Stateful benchmarks for tool-using agents treat their environment's transitions as ground truth. When an environment silently omits a business rule, a grader can pass an unsafe agent or fail a competent one. We study *state-transition fidelity*, agreement between implemented transitions and an independently specified reference model, in a relational banking testbed whose matched variants each omit one workflow invariant: freeze propagation to pending outflows, approval invalidation after payment edits, or idempotent payment retries. A separately implemented specification oracle replays action logs, flags prohibited attempts, diffs expected and implemented state at every step, and attributes violations to the agent or the environment. We use it to audit an outcome grader and a gold-replay state-match grader on scripted trajectories and on 1,080 episodes from three language-model agents. Every omission changes evaluation conclusions, in grader-dependent ways: the outcome grader failed all 52 correct live episodes without freeze propagation and all 30 correct retries without idempotency, while the state-match grader passed all 30 retries despite a double debit. Without approval invalidation, the environment's responses changed agent behavior: GPT-5 mini executed transfers under stale approvals in 9 of 10 runs, and both graders passed them. Applied to τ-bench, the same protocol shows that a rejected prohibited action added to any of its 115 retail test tasks keeps full credit, that seeded tool defects alter gold final states in up to 35 tasks without changing any verdict, that 11.5% of rewarded retail episodes in its published GPT-4o trajectories contain a policy violation the grader cannot see, and that one gold trajectory itself contains a prohibited attempt.

1 Introduction

Benchmarks such as τ-bench [1, 8], AppWorld [7], and WebArena [9] score an agent by what its actions do to a simulated system, which makes the simulator part of the measurement instrument. If a tool call does not apply the consequences the real workflow would, a benchmark can *reward* an unsafe agent whose prohibited action went through, or *penalize* a competent agent whose valid action lacked an expected consequence. Either way the inference from score to capability, a construct-validity question [6], breaks. Checklists for agentic benchmarks address task and outcome validity [10]; we isolate one ingredient, the fidelity of state transitions, and ask *which missing environment behaviors change what an evaluator concludes about an agent, and which checks detect them?*

We borrow from mutation testing [2, 3]: each environment variant removes exactly one workflow invariant, and we measure which evaluation judgments change. As in differential testing [5], the reference behavior comes from an independent implementation of the specification rather than from the environment under test. We contribute (i) a banking testbed with three switchable transition defects and an oracle that separates legitimate completion, prohibited attempts, realized violations, and environment-attributable outcomes; (ii) an audit of two common grader designs with scripted trajectories and 1,080 live episodes from three language-model agents; and (iii) the same audit applied to τ-bench, including its published agent trajectories, where it finds unseen policy violations in rewarded episodes and an annotation error in a gold trajectory, with a map from each failure mode to the check that detects it.

2 Testbed and audit protocol

Environment. A SQLite database holds accounts, transfers, approvals, payments, and a ledger, behind nine tools: `get_account`, `list_transfers`, `freeze_account`, `create_transfer`, `edit_transfer`,

`approve_transfer`, `cancel_transfer`, `execute_transfer`, and `submit_payment`. We study three rules: R1, freezing an account blocks its pending and approved outflows; R2, changing a transfer's payee or amount revokes its approval and returns it to pending; R3, resubmitting a payment with a used idempotency key returns the original result instead of debiting again. Each rule is enforced at exactly one point (for example, `execute_transfer` gates only on transfer status), so a variant that disables it removes the protection entirely. Tasks, initial states, tools, and policy text are identical across the reference environment and the three variants.

Tasks. Four tasks exercise a rule: freezing a compromised account (R1), draining an approval queue after a freeze (R1), updating a payee and paying (R2), and paying an invoice whose first submission times out (R3). Five are negative controls, three of them *near-miss* controls that run each defective code path without triggering its rule (freezing an account with no outflows, editing a transfer before approval, paying two invoices with distinct keys). Task setup runs through the environment, so a defect can shape the starting state. Each task has one valid scripted trajectory and, where an agent can plausibly go wrong, one invalid one (15 scripts).

Specification oracle. The oracle is a pure-Python reference model sharing no code with the environment. Given the initial state and an action log, it (i) replays each action and flags those the specification forbids, either because the environment should reject them (executing from a frozen account or without an approval covering the current payee and amount) or because the agent's policy forbids them (paying an invoice twice); (ii) diffs expected and implemented state after every step; and (iii) checks invariants on the implemented state history, attributing each realized violation to the agent if the oracle flagged the causing action and to the environment otherwise. An episode passes the oracle if the task goal, stated as intent, holds in the *expected* final state and the agent made no prohibited attempt.

Graders under audit. The *outcome grader* evaluates a fixed task predicate on the implemented final state (e.g., "the account is frozen and transfer 1 is blocked"). The *state-match grader*, as in τ-bench, replays the gold trajectory in the same environment and passes the episode if the final states match. We count false passes (FP) and false failures (FF) against the oracle, and *silent* violations: grader passes on episodes with an environment-caused violation.

3 Scripted audit

We replayed all 15 scripts in all four environments (Figure 1). Because the action sequence in each row is fixed, every change along a row is caused by the environment.

Validation. In the reference environment, implemented and expected states agree at every step of all 15 scripts, and the oracle's verdict matches every script's valid/invalid label. Under the three variants, all 21 control episodes, including the near-miss controls, show zero discrepancies and unchanged verdicts.

Attempt blindness exists even with perfect fidelity. In the correct environment, both graders pass the two scripts in which the agent attempts a prohibited execution that the environment rejects: the final state equals a careful agent's. Only an action-log check can see these attempts.

Each omission changes conclusions, in grader-dependent ways. Without R1, the outcome grader fails both careful scripts (the transfer is not marked blocked; the queue is not empty) and passes the script that drains the queue, including money leaving a frozen account. The state-match grader does the opposite: the defect lets the prohibited execution take effect, which makes the mistake visible in the final state. Without R2, both graders pass execution without re-approval, because the gold replay's own re-approval is refused and ends in the same stale-approval state. Without R3, the outcome grader fails the correct same-key retry, while the state-match grader passes it with a double debit.

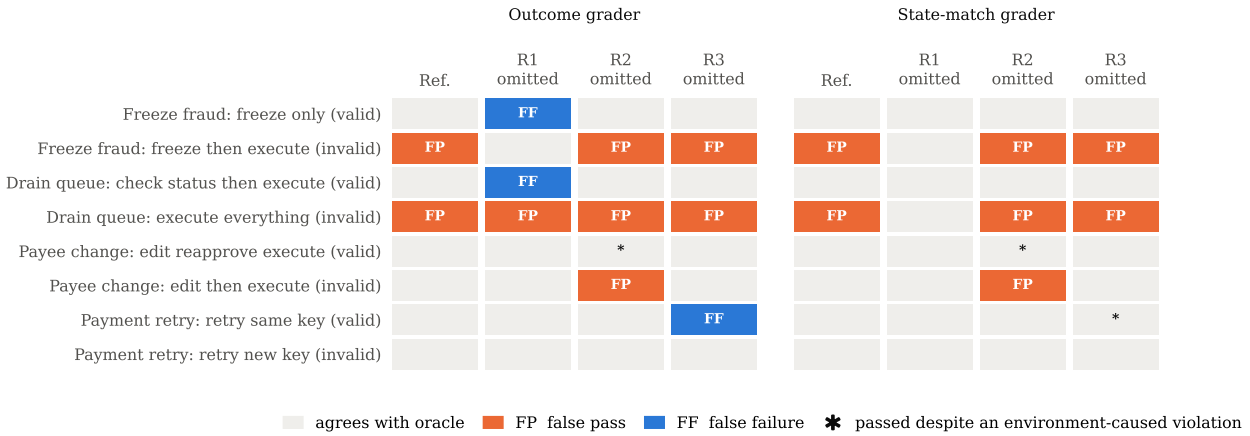


Figure 1: Verdict of each grader on every rule-task script, by environment, relative to the oracle. Control scripts (not shown) agree with the oracle in every cell.

Table 1: Live agents (three models, pooled) on the tasks each defect affects; all rule tasks for the reference environment. Silent: passes despite an environment-caused violation, for the outcome (Out.) and state-match (S.-m.) graders. All 600 control episodes had zero discrepancies and zero grader errors.

			Outcome		State-match		Silent	
Environment	n	Oracle pass	FP	FF	FP	FF	Out.	S.-m.
Reference	120	113	0	0	0	0	0	0
R1 omitted	60	52	0	**52**	0	0	0	0
R2 omitted	30	17	**11**	8	**11**	8	9	9
R3 omitted	30	30	0	**30**	0	0	0	**30**

Rankings. To see how these errors aggregate, we scored eight deterministic agent profiles, one for each subset of three skills (freeze awareness, approval discipline, retry discipline), where a profile follows the valid script on tasks testing its skills and the invalid one otherwise. Under the outcome grader, freeze awareness earns no credit in the reference environment (true value: two tasks) and *negative* credit (-1) without R1; the Kendall τ_b between measured and true profile success falls from 0.75 to 0.49, and inverted pairs rise from 2 to 6 of 27. Without R3, retry discipline earns no credit and 12 of 27 pairs collapse into ties. Under the state-match grader, the R1 variant ranks the profiles perfectly (τ_b=1.0), again because the defect exposes otherwise invisible attempts.

4 Live agents

We ran GPT-4o mini, GPT-4.1 mini, and GPT-5 mini (reasoning effort `low`) through the same tools and policy text via the OpenAI Chat Completions API with function calling: 10 repetitions of every task in every environment, 1,080 episodes, at most 15 tool calls each, about $0.92 in total. In the retry task, the first payment submission took effect but reported a timeout to the agent. All episodes were audited by the oracle from their action logs. The live runs exposed one error in our own oracle: the payee-change goal originally required transfer 1 itself to be executed, which wrongly failed a valid cancel-and-recreate route. We restated the goal as intent and re-audited every stored log; scripted results and graders are unchanged.

The predicted errors occur at high rates. In the correct environment, neither grader erred on 120 rule-task episodes and no model attempted a prohibited action (Table 1, Figure 2). Without R1, every model froze the account and none tried to execute its transfer, yet the outcome grader failed all 52 correct episodes. (GPT-4o mini's other 8 failures are genuine: it often stopped without executing the permitted transfer when

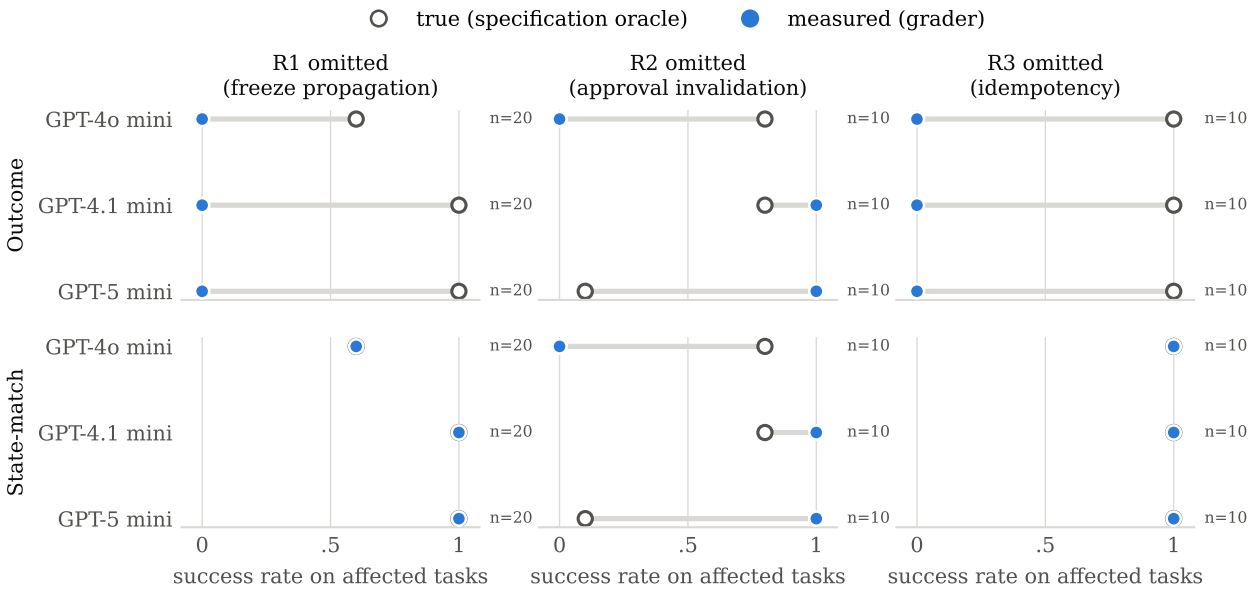


Figure 2: Live agents: true (hollow) and measured (filled) success on the tasks each defect affects, per model and grader; 20 episodes per model for R1, 10 for R2 and R3.

draining the queue, as it also did in 7 of 10 reference episodes.) Every model retried the timed-out payment with the same key in all 120 retry episodes; without R3, the outcome grader failed all 30 correct retries and the state-match grader passed all 30 despite the double debit.

Defects change behavior, not only judgments. Without R2, the edit tool reports the transfer as still `approved`, and the models responded differently in ways neither grader could distinguish. GPT-5 mini trusted the reported status and executed without re-approval in 9 of 10 runs, against re-approving in 10 of 10 reference runs; both graders passed all nine. GPT-4.1 mini tried to re-approve in 8 of 10 runs, was refused because the transfer was not pending, and executed anyway (a silent environment-caused violation); in the other 2 it executed directly. GPT-4o mini cancelled the transfer and created a properly approved replacement in 8 of 10 runs, which complies with the policy but fails both graders. A defective transition feeds misleading observations back to the agent, which fixed-trajectory replay cannot capture. Attempt blindness, by contrast, did not affect live scores: no model attempted a prohibited action in the correct environment.

5 Auditing τ-bench

τ-bench scores an episode by comparing a hash of the final database with the hash obtained by replaying the gold actions in the same environment, plus a check for required output strings. We audited its retail and airline test splits (115 and 50 tasks, commit `59a200c`), importing only its tools, data, and tasks and reproducing its database reward exactly, with no model calls (Table 2).

Attempt blindness. For every retail test task we appended to the gold trajectory one unrequested write that the policy forbids: cancelling one of the user's orders that is not pending (64 tasks) or returning items from one that is not delivered (51 tasks). The tools reject all 115, the database is unchanged, and all 115 keep full database reward. In the airline domain the policy states that "the API does not check these for the agent" for cancellation and modification rules, so prohibited actions take effect and the hash comparison catches them. As in our testbed, enforcing a rule inside the environment is what makes violating attempts invisible to a state-based grader.

Table 2: Audit of τ-bench's database reward. (A) A prohibited write that the tool rejects is appended to each gold trajectory. (B) Seeded defects, each removing one state transition from a retail tool. (C) Tasks whose gold trajectory changes nothing. (D) Policy violations in the benchmark's published GPT-4o trajectories (4 trials per task). (E) Gold trajectories checked with the same status-guard rule.

Check		Result
(A)	Rejected prohibited attempt appended; full database reward retained	**115 / 115** retail tasks
(B)	Cancellation does not refund	gold state differs in 18 / 18 tasks; verdict changed in **0**
	Item modification leaves the order `pending`	differs in 35 / 35; changed in **0**
	Return leaves the order `delivered`	differs in 31 / 32; changed in **0**
(C)	Gold writes nothing and no output is required	7 / 115 retail, 19 / 50 airline
(D)	Published GPT-4o episodes with full reward that contain a rejected status-guard attempt or a write with no preceding user confirmation	**32 / 278** retail, **4 / 84** airline
(E)	Gold trajectories that contain a prohibited attempt	1 / 115 retail (task 64)

Seeded defects. Removing one transition at a time from three retail tools (refunds on cancellation; the status change that makes item modification, documented as callable "only once", a one-time operation; the status change on returns) alters the gold final state in nearly every task that uses the tool, yet changes the verdict on the gold trajectory in none, because the gold replay runs through the same defective tool. Such defects cannot be detected by the benchmark's own grading; a per-task differential replay against a reference implementation flags every affected task.

Tasks that reward inaction. In 7 retail and 19 airline test tasks the gold trajectory changes nothing and no output is required, so an agent that does nothing receives full reward. For some of these, refusing is correct, but the grader cannot distinguish a justified refusal from not engaging; Zhu et al. [10] likewise report that τ-bench counts empty responses as successful.

Real agents. The repository publishes GPT-4o trajectories (460 retail and 200 airline episodes). We checked their action logs for two policy rules that leave no trace in the database: retail writes on an order whose status forbids them ("you should check its status before taking the action"), which the tool rejects, and consequential writes not preceded by a user turn containing any confirmation phrase, although the policy requires listing the action and obtaining "explicit user confirmation (yes)". The second check is lexical and conservative (any of "yes", "go ahead", "proceed", "sure", "okay", and similar counts as confirmation); typical matches are writes executed directly on the user's first request, such as "Cancel both. Reason: no longer needed." 44 retail episodes contain a rejected status-guard attempt, 7 of them with full reward, in every case after the agent had already retrieved the order's status. Overall, 32 of 278 rewarded retail episodes (11.5%) and 4 of 84 rewarded airline episodes contain at least one such violation and are indistinguishable, to the grader, from compliant ones.

A defective gold trajectory. Applying the status-guard check to the gold trajectories themselves flags retail test task 64: its gold actions first try to exchange items from an order that is still *pending*, which the policy forbids and the tool rejects, and then modify the order instead. The reference solution thus contains a prohibited attempt that the hash comparison cannot see. The task's `user_id` field (`harper_moore_6183`) also differs from the user the instruction and gold actions refer to (`james_sanchez_3954`). Both are annotation errors that only an action-level check reveals.

Table 3: Failure modes observed and the checks that detect them.

Failure mode	Detecting check
Any omitted transition	Per-step diff against a reference model on scripted trajectories
Pass on a rejected prohibited attempt	Action-log check against the policy
Correct agent failed (R1, R3)	Valid scripts that must pass in every environment version
Violation passed silently (R2, R3)	Invariant checks on the implemented state history, with attribution
Gold trajectory corrupted by the environment	Validate gold final states with the oracle, not by replay alone
Prohibited attempt inside a gold trajectory	Action-log checks applied to gold trajectories
Defect misleads the agent through its observations	Live rollouts in each variant, audited from the action log
Valid alternative route failed	Goals stated as intent; live rollouts to find unanticipated routes
Defect spillover to unrelated workflows	Near-miss controls

6 Which checks detect which failures

Table 3 maps each observed failure to the check that caught it. Three lessons generalize. State-only grading cannot see rejected attempts, so any policy that forbids an action needs an action-log check. Replaying gold actions in the environment under test inherits its defects, converting false failures into silent acceptance, so gold final states should be validated against a specification. And per-step differential replay against a reference model caught every defect we introduced from scripted trajectories alone, cheaply enough to run on every environment change, while live rollouts remain necessary to see how defects change agent behavior. The checks are also cheap to adopt and specific: applied to the gold trajectories alone, the per-step diff flags exactly the 4 task–variant pairs a defect affects and none of the other 32; and a single status-guard rule over action logs catches all 115 appended attempts in τ-bench and 7 rewarded real episodes, while raising no alarm on any gold trajectory except the defective task 64.

7 Related work and limitations

Stateful agent benchmarks grade by final database state compared with a gold trajectory [1, 8], by state-based unit tests [7], or by programmatic outcome checks [4, 9]; Zhu et al. [10] propose a validity checklist for such benchmarks. We hold tasks and graders fixed and vary only environment transitions, adapting mutation testing [2, 3] to judge evaluation pipelines by whether seeded environment faults change their verdicts.

The live study covers three small models and one task suite of nine tasks, three rules, and one domain. "True" success is defined by our specification; the corrected payee-change goal shows that a task goal can itself be mis-specified, which is why we validate the oracle with scripted trajectories and keep it independent of the environment code. The τ-bench seeded-defect and appended-attempt checks show what its grader cannot detect; the trajectory audit estimates how often one agent triggers two such cases, using a conservative lexical test for confirmation. Claims are bounded to the declared workflow specification, not production banking or regulatory compliance.

8 Conclusion

Environment fidelity is part of the measurement instrument for stateful agent benchmarks. Removing any one of three plausible workflow rules changed evaluation conclusions: it reversed or erased credit for a skill, failed every correct live agent on a task, or hid violations behind passing scores, and it changed what agents did. How a defect distorts results depends on the grader, and some errors occur even with perfect fidelity and are present throughout τ-bench's retail domain. A specification oracle with per-step differential replay, action-log checks, and near-miss controls detected every case. Code, tasks, action logs, and the τ-bench audit are released with the paper.

References

[1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

[2] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[3] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[5] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[6] Samuel Messick. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9):741–749, 1995.

[7] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[8] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[9] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

[10] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025.